

Opportunity Radar

A phase-by-phase build plan for an Apify Actor that matches African students against real scholarship, grant, and admission opportunities — and verifies each one isn't a scam before it reaches the user.

Theme: Ship & Earn Africa

Team of 3

Stack: Node.js + Crawllee

LLM: Claude / Gemini

Trust signal: Google Custom Search

PHASE 00

Setup

30-60 min

Get the team from zero to a runnable Actor skeleton, and split ownership before anyone starts writing scraping logic.

TASKS

- Create an Apify account and install the Apify CLI
- Scaffold a new Actor project (Node.js + Crawllee template)
- Create the GitHub repo and share access with both teammates
- Split ownership: Scraping · Matching & LLM logic · Trust-scoring & API integrations

Output: An empty Actor that runs on Apify's cloud and returns a placeholder dataset. Nothing built yet — this just proves the pipes are connected.

PHASE 01

Single Scraper, Raw Output

Core proof of life

Prove the Actor can pull real, structured data from a real site before any intelligence layer is added.

TASKS

- Build one scraper targeting PhDportal's scholarship search page
- Input: field of study + country filter only
- Collect raw listings — title, link, deadline, description text
- Write results to an Apify dataset

Output: A dataset of real, unprocessed scholarship listings scraped live from PhDportal.

PHASE 02

Matching, Powered by an LLM

The differentiator

Move past keyword filtering — reason about whether a listing's messy real-world eligibility text actually fits a specific person.

TASKS

- Expand input to a full user profile
- Send each listing's text + the user profile to the LLM
- LLM returns structured eligibility judgment, not just a keyword match
- Keep a cheap regex pre-filter for hard facts (deadline already passed = auto-reject, no LLM call needed)

INPUT SCHEMA

```
{
  "educationLevel": "PhD | Masters | Bachelors",
  "fieldOfStudy": "string",
  "country": "string",
  "fundingNeeded": true,
  "gpaOrGrade": "optional string"
}
```

LLM ADDS TO OUTPUT

```
{
  "eligibilityMatch":
    "Eligible | Partial | Not Eligible",
  "missingRequirements": ["string"],
  "reasoning": "one-line string"
}
```

Output: Every listing now says whether this specific user qualifies, what's missing, and why — not just whether keywords overlapped.

PHASE 03

Trust Scoring, With Evidence

The safety layer

Answer "how do you know it's real?" with actual external evidence, not just a text-pattern guess.

PIPELINE PER LISTING



SIGNALS THE LLM WEIGHS

- Requires upfront payment (processing fee, "refundable deposit," gift cards)
- Vague or absent eligibility criteria ("open to everyone, apply now")
- Artificial urgency disproportionate to the opportunity's size
- Sponsoring org has no independent presence found via Google Custom Search
- Listing appears nowhere else on the web besides its original source

Verified

Caution

High Risk

Seed 2–3 realistic fake listings by hand for the demo — safer and more controllable than live-scraping scam sources, and it guarantees a "High Risk" catch happens on stage.

PHASE 04

Second Source + Polish

Breadth & cleanup

Widen coverage beyond PhDs and tighten the output before it's demo-ready.

TASKS

- Add Opportunity Desk as a second scraper, merged into the same output schema
- Add basic error handling for missing/malformed fields
- Stretch goal only, if time remains: a minimal front end (simple form or Next.js on Vercel) calling the Actor via the Apify API

Output: One unified feed spanning PhD-specific and general African-opportunity sources, resilient to messy real-world data.

PHASE 05

Demo Prep

Final stretch

Land the two-minute arc that actually convinces a judge, live.

TASKS

- Script a live run: enter a realistic student profile
- Show a genuine match with clear reasoning
- Show a seeded fake listing get caught as High Risk, with reasons shown
- Tie the close back to the theme: the Actor is publishable to the Apify Store, so it keeps earning after the hackathon ends

Output: A rehearsed, two-minute demo that shows both pillars — match and verify — in one continuous story.